

INTERNAL EXPLAINER • 3-4 MIN

Book Recommendation RAG

TypeScript, LangGraph, Groq, Voyage AI, ChromaDB - architecture, LLM cost points, and a live demo.

One domain bot, five pieces



TypeScript

End to end, one language



LangGraph

Orchestration as a state graph



Groq

LLM inference, fast + cheap



Voyage AI

Embeddings for retrieval

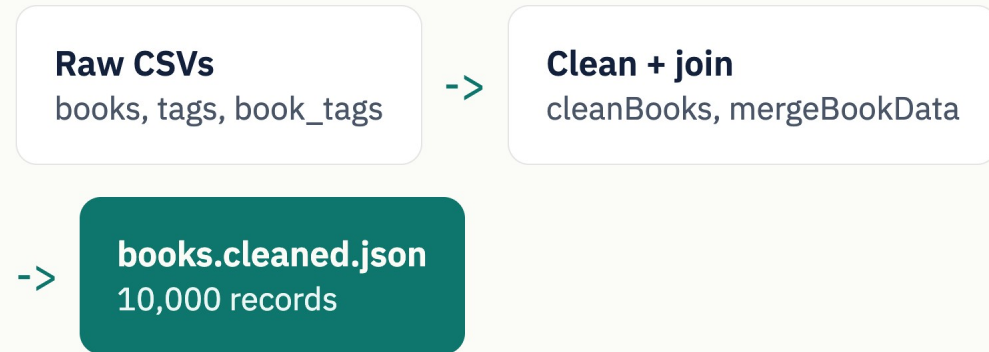


ChromaDB

Local vector store

Both the LLM and the embedding model sit behind a small interface - ChatModel, EmbeddingProvider - swappable without touching callers.

Goodbooks-10k, one document per book



No synopsis or review text in this dataset - just structured metadata. That's why there's no fixed-char or paragraph chunker: each record already is one small, atomic unit.

```
documents.ts - buildBookPageContent
```

```
Title: The Hunger Games
```

```
Author: Suzanne Collins
```

```
Publication Year: 2008
```

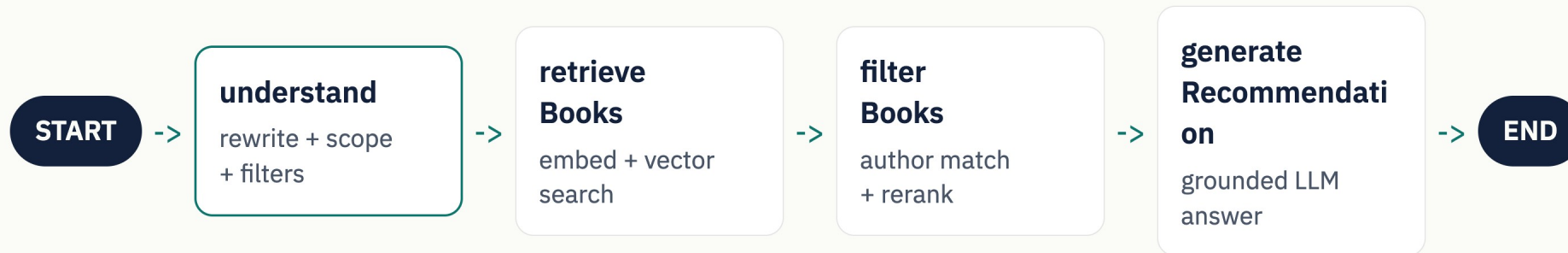
```
Language: eng
```

```
Tags: dystopian, ya, sci-fi
```

```
Average Rating: 4.34
```

```
Ratings Count: 4780653
```

A LangGraph state machine, not one prompt



understand also short-circuits straight to END when the request is out of scope - no retrieval, no second LLM call.

Conversation memory: LangGraph's own **MemorySaver** checkpoint, keyed by **thread_id** - not hand-threaded state.

Each node, one job

Node	Responsibility	Cost / turn
understand	Rewrite follow-up into standalone query, scope-check, extract structured filters - one call	1 Groq call
retrieveBooks	Embed the resolved query, semantic search in Chroma, ANDed with the structured-filter where clause	1 Voyage call
filterBooks	Fuzzy author match client-side, then rerank (no-op seam today - free)	no LLM/embed
generateRecommendation	Generate the final answer, strictly grounded on retrieved candidates only	1 Groq call

Out-of-scope requests short-circuit after understand - no retrieval or generation cost at all.

Prompt to query: hardcoded, then LLM

BEFORE

Regex + keyword rules

- Couldn't tell negation from the real ask ("not fantasy" vs "fantasy")
- Couldn't tell a follow-up from a topic shift across turns
- New phrasing kept finding new edge cases to patch

AFTER

One LLM call - understandNode

- Sees full history + new message together, one structured JSON response
- Rewrites into a standalone query, checks scope, extracts filters - all at once
- Fails safe: bad JSON falls back to plain semantic search, never blocks the turn

Why one call, not three: only a model that reads the whole conversation can tell "still depends on the last answer" apart from "new topic".

COST RECAP

What actually costs money

2

Groq LLM calls per turn
understand + generateRecommendation

1

Voyage embedding call per turn
query embedding at retrieval

10K

embeddings, one-time
ingest, batched 128/request

Out-of-scope turns and the no-op reranker cost nothing extra - the seam is there for a real reranker later.

One thread, five cases

#	Prompt	Proves
1	"Dystopian books after 2008"	Genre tag + year filter, ANDed with semantic search
2	"Books by J.K. Rowling"	Author fuzzy match - big candidate pool, client-side filter
3	"What about rated above 4.5?"	Follow-up - replaces the old constraint, doesn't stack it
4	"Best spy book"	Topic shift - fresh request, not limited to the last list
5	"Book me a flight to Delhi"	Out of scope - short-circuits, zero retrieval/generation cost

Same thread_id throughout - this is the checkpointer's memory doing the work.

RECAP

LangGraph nodes, one LLM call to understand, grounded retrieval, one LLM call to answer.

Next: a real reranker (Voyage rerank / cross-encoder)

Next: feedback-driven ranking, already typed and seamed in